

1. You want to create a model with a variable number of layers, so you store them in a Python list. What is the primary problem with this approach? 1 point
- ☐ The model will run too slowly
 - ☐ You can't iterate through Python lists in the forward method
 - ☒ The layers won't train or save
 - ☐ Layers stored in lists can't be moved to GPU with .to(device)

2. How does PyTorch handle backpropagation when a model uses conditional execution (if-statements in forward)? 1 point
- ☐ It averages the gradients from all possible branches
 - ☒ It builds a fresh computation graph for whichever branch was taken
 - ☐ Conditional execution isn't possible—models must always run the same layers
 - ☐ It requires you to manually specify which branch was taken

3. In a Siamese network, parameter sharing means that both branches apply exactly the same convolutional layers to two images. How does this affect training? 1 point
- ☐ It disables contrastive loss margins
 - ☒ Gradients from both branches update the same shared weights
 - ☐ It doubles the number of learnable parameters, reducing overfitting
 - ☐ It prevents back-propagation through the second branch

4. What is a residual connection (skip connection) in a ResNet block? 1 point
- ☐ A connection that only activates when gradients become too small
 - ☐ A separate pathway that processes the input through additional convolutional layers
 - ☐ A mechanism that selectively keeps or discards features based on their importance
 - ☒ A direct addition of the original input to the block's output

5. What does the growth rate control in a DenseNet? 1 point
- ☒ How many new channels each dense layer adds
 - ☐ The compression factor used in transition layers
 - ☐ The size of the spatial dimensions after each layer
 - ☐ How many dense blocks are in the network

6. What architectural pattern does this class illustrate? 1 point

```
1 class SelectiveForward(nn.Module):
2     def __init__(self):
3         super().__init__()
4         self.small = nn.Sequential(nn.Linear(100, 50), nn.ReLU())
5         self.big = nn.Sequential(nn.Linear(100, 200),
6                                 nn.ReLU(),
7                                 nn.Linear(200, 50))
8     def forward(self, x, hard_example: bool):
9         if hard_example:
10            return self.big(x)
11        else:
12            return self.small(x)
```

- ☐ Skip connections
- ☐ Parameter sharing
- ☐ Dynamic model creation
- ☒ Conditional execution

7. During backpropagation through a residual block, why does the gradient include an extra additive term? 1 point
- ☐ It helps balance gradient magnitudes across different layers
 - ☐ It allows the network to skip poorly performing layers during training
 - ☐ It amplifies small gradients to speed up training
 - ☒ It provides a direct path for gradients to flow backward without vanishing

8. Insert the single layer marked # complete code here so the TransitionLayer will 1 point

- batch-norm + ReLU the input
- compress the channel count by compression_factor
- down-sample the spatial size by 2×

```
1 class TransitionLayer(nn.Module):
2     def __init__(self, in_channels, compression_factor=0.5):
3         super().__init__()
4         self.bn = nn.BatchNorm2d(in_channels)
5         out_channels = int(in_channels * compression_factor)
6
7         # complete code here
8         self.pool = nn.AvgPool2d(kernel_size=2, stride=2)
9
10    def forward(self, x):
11        x = F.relu(self.bn(x))
12        x = self.conv(x)
13        x = self.pool(x)
14        return x
```

- self.conv = nn.Conv2d(in_channels, out_channels, kernel_size=1, stride=1, bias=False)
- self.conv = nn.Conv2d(out_channels, in_channels, kernel_size=1, bias=False)
- self.conv = nn.Linear(in_channels, out_channels)
- self.conv = nn.Conv2d(in_channels, out_channels, kernel_size=3, stride=2, padding=1, bias=False)

9. In the *ResidualBlock* used for ResNet, when must the optional downsample layer be supplied so that the skip-connection tensor can be added to the main path?

1 point

- ☐ Never—the addition works for every combination of stride and channel sizes.
- ☐ Only when `in_channels < out_channels` and the stride equals 1.
- ☒ Whenever either `stride != 1` or `in_channels != out_channels`.
- ☐ Only when `stride == 1` and `in_channels == out_channels`.

10. DenseNet concatenates each layer's output to all previous feature maps, causing the channel count to grow. Which built-in mechanism keeps this growth from exploding between dense blocks?

1 point

- Skip connections with element-wise addition to reduce channels automatically.
- Batch normalization to drop exactly 50% of channels during training.
- Shared weights to force later layers to reuse earlier channels instead of allocating new ones.
- A TransitionLayer with 1x1 convolution to reduce channels